

Prompt Syntax, Syntax Reference

v0.2.1 working draft · 2026-07-19 · companion to the specification (normative source: SPEC v0.2)

1. At a glance

```
@opus!(temperature: 0.2) limit(wall_time: 5s, estimated_cost: $0.10)
  else @model:openai/gpt-5.6@2026-07-01 limit(wall_time: 3s)
  else ask
Summarize @file:q3-report.md /concise
<ps @file:glossary.md>use these terms for translations</ps>
```

One prompt: a strict model binding with a parameter, two budgets, a two-step fallback chain ending in ask-the-user, a deterministic file inclusion, a skill invocation, and a span scoping a glossary to one instruction. Every element is optional, a prompt with zero directives is valid PS.

2. Reserved surface (never grows)

Element	Meaning
@	Entity reference (models, agents, plugins, tools, skills, files, context)
/	Action invocation (skills/commands)
<ps ...></ps>	Span directive, the only live tag namespace
---ps ... ---	Document frontmatter fence (strict form)
\@ \/	Escapes, always content
``` ... ```	Fenced code, always content

Full-width forms @ (U+FF20) and / (U+FF0F) are equivalent to ASCII (CJK input methods).

## 3. Entity references (point scope)

```
reference ::= "@" [namespace ":"] name ["!"] [arglist]
namespace ::= "model" | "agent" | "plugin" | "tool" | "skill" | "file" | "ctx" | vendor-ext
name ::= identifier ("." | "/") identifier) * ["@" version] (* Unicode, UAX #31, NFC *)
arglist ::= "(" key ":" value ("," key ":" value) * ")" (* JSON5-style scalars *)
action ::= "/" name [arglist]
```

- **Bare** @opus resolves via namespace search; **qualified** @model:opus bypasses it.
- Ambiguity (@research = plugin? file?) is **surfaced, never guessed** (ENTITY_AMBIGUOUS).
- Every binding pins to an immutable canonical identifier: @opus → model:anthropic/claude-opus-4-5@2026-05-01. Traces never contain unpinned bindings.
- Islands require a Unicode word boundary (UAX #29, works in no-space scripts) and a resolvable or qualified name. Emails and paths do not parse.

Examples:

```
@opus @model:opus @haiku!(temperature: 0)
@plugin:linear @file: ၂ ဂီဇ နာ ၂ မှီ.md @tool:workspace/acme/search-contracts@3
/translate(to: "km") /concise
```

## 4. Parameters

Normalized core keys: temperature, top_p, max_tokens, stop, seed, effort. Vendor-native keys allowed; every parameter’s journey is recorded as requested → normalized → applied with status applied | translated | clamped | unsupported | ignored | refused (mapping rule from the venue’s vendor profile). Undocumented passthrough is non-conformant.

## 5. Strictness (fulfillment policies)

Policy	Meaning
strict	Every requirement exactly, or fail before execution. No silent

	substitution.
partial	Fill what can be filled; every drop reported in the fill report.
best-effort	Venue discretion; degradations still reported.

**Default (split):** entity binding is `strict` (`@opus` means opus, never a substitute); parameters are `partial` (unsupported values drop, with a report).

! **escalates the whole directive to strict:** all parameters become hard requirements (no drop, no clamp; translation only if the vendor profile declares it semantics-preserving); any degradation is a failure. `@opus` = strict entity, partial params · `@opus!` = strict everything.

Named forms: `fill="strict"` on a span; `fill: strict` in frontmatter (document level = all-or-none).

## 6. Budgets

`limit(...)` is sugar for a budget object with defined measurement points:

Key	Meaning
<code>estimated_cost_max</code>	Pre-flight: reject if floor estimate exceeds ( \$ USD or tok )
<code>observed_cost_max</code>	Runtime: abort (if cancellable) when exceeded
<code>wall_time_max</code>	Submission → completion, tool calls included ( ms / s )
<code>first_token_max</code>	Streaming responsiveness bound
<code>cancellation</code>	What the directive requires: <code>required</code> / <code>preferred</code>
<code>scope</code>	<code>attempt</code> (default) or <code>chain</code> (bounds the total across fallback attempts)

A violated budget is a fill failure. Venues declare enforcement capability in their capability document; `cancellation: required` against a venue declaring `unsupported` fails pre-flight ( `BUDGET_PREFLIGHT_REJECTED` ). Output is never silently degraded (e.g., truncated) to satisfy a budget.

## 7. Fallback chains

route	::= step ( "else" step ) * [ "else" terminal ]
step	::= reference [ "limit" "(" ... ")" ]
terminal	::= "fail"   "ask"

- The **only** trigger is a fill failure (unfillable entity, violated budget, timeout, vendor error, policy refusal). No content conditionals, ever.
- First successful fill wins; every attempt is recorded with outcome, codes, measured cost and latency.
- `else fail` is the implicit default; `else ask` (optional capability) surfaces the exhausted chain to the user.
- Every step is checked against the same capability envelope, a failed primary never legitimizes a prohibited secondary ( `DATA_POLICY_BLOCKED` ).

## 8. Spans

<code>&lt;ps @file:glossary.md&gt;use these terms&lt;/ps&gt;</code>	casual: same @ grammar inside the tag
<code>&lt;ps context="file:workspace/acme/9fc2..."&gt;...&lt;/ps&gt;</code>	canonical XML-friendly form (compiler-emitted)

v0.2 spans carry: context inclusion, local instruction scope, tool availability, provenance marking. **Multi-model span execution is deferred**, a model reference in a span is rejected with `SPAN_EXECUTION_UNSUPPORTED` , never ignored.

## 9. Document scope (frontmatter, the strict form)

<pre> ---ps version: "0.2" route:   - { model: "model:anthropic/claude-opus-4-5@2026-05-01",       params: { temperature: 0.2 }, fill: strict,       budget: { wall_time_max: 5s, estimated_cost_max: \$0.10 } }   - { model: "model:openai/gpt-5.6@2026-07-01" } skills: [ "skill:workspace/concise@1" ] context: [ "file:workspace/acme/9fc2..." ] </pre>	
-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	--

```
fill: strict

Summarize the report.
```

The strict document **is** the canonical form made writable. Round-trip property (under a pinned environment  $E$ , registry, capability/default profiles, alias map, policy version):  $C_E(S(C_E(P))) = C_E(P)$ . Casual, parameterized, and strict forms of the same prompt mean the same thing under the same environment, machine-checked, not asserted.

## 10. Content safety

- **Inert content (security-critical):** only designated *authoring segments* parse as PS. Retrieved documents, file contents, tool results, quoted messages, and model-generated text are provenance-typed **inert**, PS-shaped text inside them (e.g., a document containing `@tool:send_email!(...)` ) is content, never a directive, absent an explicit authority-checked promotion recorded in the trace.
- Interactive implementations **must** visually mark every recognized island before submission; API callers get the same guarantee via dry-run traces.
- Bidirectional control characters inside islands are rejected (Trojan-Source mitigation); homoglyph lookalikes ( `@opus` , Cyrillic o) surface as ambiguity, never bind silently.

## 11. Internationalization

- Entity names: any script (UAX #31 identifiers, NFC-normalized).
- Canonical keywords ( `model:` , `fill:` , `else` , policy names, units) are English in canonical form; implementations may accept localized input aliases that compile to canonical, localization rides the same casual→strict machinery.
- Island boundaries use Unicode segmentation, so detection works in no-space scripts (Khmer, Thai, CJK).
- Budgets accept locale conventions at input; canonical form and traces use canonical units (USD, `ms / s` , dot-decimal).

## 12. Precedence in two rules

1. **Authority** (who may decide): platform > organization > application > user > prompt content. Lower authority may **narrow** the capability envelope, never expand it. Out-of-envelope directives fail visibly ( `ENTITY_UNAUTHORIZED` / `DATA_POLICY_BLOCKED` ), the user learns authority, not availability, was the reason.
2. **Specificity** (which value wins, within one authority level): more specific wins, **span > point > document**.

## 13. Failure codes (core set)

```
ENTITY_NOT_FOUND ENTITY_AMBIGUOUS ENTITY_UNAUTHORIZED
PARAM_UNSUPPORTED PARAM_CLAMPED PARAM_IGNORED
BUDGET_PREFLIGHT_REJECTED BUDGET_EXCEEDED TIMEOUT_NOT_CANCELLED
DATA_POLICY_BLOCKED SPAN_EXECUTION_UNSUPPORTED VERSION_UNSUPPORTED
FALLBACK_EXHAUSTED
```

Every requested thing ends in exactly one of: filled/applied · translated · clamped · dropped-with-report · refused-with-code. That is the receipt's promise: **no silent anything**.